STAT340 Lecture 09: Multiple Linear Regression

Brian Powers

Updated 11/5/2025

Introduction I

Last week, we discussed simple linear regression, wherein we predict a *response* Y based on a single *predictor* X, according to the model

$$Y = \beta_0 + \beta_1 X.$$

In lecture, we had an extended discussion of predicting crime rates (on a 22 year lag) based on levels of lead in the atmosphere. That is, our predictor was lead level (measured in metric tons) and our response was aggravated assaults per million people. We predicted a *single* response from a *single* predictor.

In reality there are lots of things beyond lead levels that might have an effect on crime rates. Examples include availability of social services, amounts of green space in a city, population density, income inequality...

How might we build a model that uses *multiple* predictors, rather than a single one, to predict our response? Well, this is precisely the idea behind *multiple linear regression*, our focus this week.

Multiple regression I

Everything so far is likely (mostly) familiar to you from STAT240: regressing one variable against another. What happens, however, when we want to incorporate multiple different predictors in our model?

Multiple regression II

Example: salary and education There are many other factors (college major, demographic information, parents' level of education, etc.) that might predict salary in addition to simply years of education, and that their inclusion in our model would improve our predictive accuracy.

Example: housing prices Suppose we want to predict, based on what we know about a house, how much that house is likely to sell for. One approach would be to predict housing price based on a collection of information such as square footage, age of the house, number of bedrooms, proximity to parks, etc.

Specifying multiple predictors I

We just add more predictors (and more coefficients) to our linear function. If we have p predictors plus an intercept, we predict the response y according to

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \cdots + \hat{\beta}_{p-1} x_{p-1} + \hat{\beta}_p x_p,$$

where $x_1, x_2, \ldots, x_p \in \mathbb{R}$ are predictors.

Specifying multiple predictors II

Similarly, our model now takes the form that for each $i = 1, 2, \ldots, n$, we observe

$$Y_i = \beta_0 + \beta_1 X_{i,1} + \beta_2 X_{i,2} + \cdots + \beta_{p-1} X_{i,p-1} + \beta_p X_{i,p} + \epsilon_i,$$

where ϵ_i is an error term (again, assumed normally distributed, independent over i, etc) and $X_i = (X_{i,1}, X_{i,2}, \ldots, X_{i,p})^T \in \mathbb{R}^p$ is a vector of predictors.

Specifying multiple predictors III

A convenient way to think about our data set, then, is to make a matrix in which each row corresponds to an observation, and each column corresponds to a predictor:

$$\mathbf{X} = \begin{bmatrix} X_{1,1} & X_{1,2} & \cdots & X_{1,p} \\ X_{2,1} & X_{2,2} & \cdots & X_{2,p} \\ \vdots & \vdots & \ddots & \vdots \\ X_{n-1,1} & X_{n-1,2} & \cdots & X_{n-1,p} \\ X_{n,1} & X_{n,2} & \cdots & X_{n,p} \end{bmatrix}$$

We can tack on a column of ones corresponding to our intercept term,

$$\mathbf{X} = \begin{bmatrix} 1 & X_{1,1} & X_{1,2} & \cdots & X_{1,p} \\ 1 & X_{2,1} & X_{2,2} & \cdots & X_{2,p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & X_{n-1,1} & X_{n-1,2} & \cdots & X_{n-1,p} \\ 1 & X_{n,1} & X_{n,2} & \cdots & X_{n,p} \end{bmatrix}$$

Specifying multiple predictors IV

and then we can write our regression formula

$$y_i = \beta_0 + \beta_1 x_{i,1} + \beta_2 x_{i,2} + \cdots + \beta_p x_{i,p} + \epsilon_i \quad \text{for } i = 1, 2, \ldots, n$$

as a matrix-vector equation,

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

where $\mathbf{y} = (y_1, y_2, \ldots, y_n)^T$ is a vector of our responses, and $\boldsymbol{\beta} = (\beta_0, \beta_1, \beta_2, \ldots, \beta_p)^T$ is a vector of our coefficients.

$\boldsymbol{\epsilon}_i$ is a vector of n iid normal random variables.

Example: the mtcars dataset I

Let's recall the `mtcars` data set, which includes a number of variables describing the specifications and performance of a collection of car brands.

```
head(mtcars)
##                    mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
## Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
## Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
## Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
```

Let's suppose that we are interested in predicting the quarter mile time (`qsec`, the time it takes the car to go 1/4 mile from a dead stop) based on its engine displacement (`disp`, measured in cubic inches), horsepower (`hp`, measured in... horsepower) and weight (`wt`, measured in 1000s of pounds).

Example: the `mtcars` dataset II

That is, we want to build a multiple linear regression model of the form

$$\text{qsec} = \beta_0 + \beta_1 \text{disp} + \beta_2 \text{hp} + \beta_3 \text{wt} + \epsilon$$

To fit such a model in R, the syntax is quite similar to the simple linear regression case. The only thing that changes is that we need to specify this model in R's notation. We do that via `qsec ~ 1 + disp + hp + wt`.

Let's fit the model in R and see what happens.

```
mtc_model <- lm( qsec ~ 1 + disp + hp + wt, data=mtcars);
```

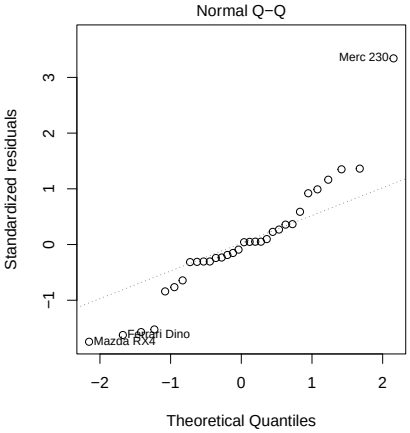
Example: the mtcars dataset III

```
##
## Call:
## lm(formula = qsec ~ 1 + disp + hp + wt, data = mtcars)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.8121 -0.3125 -0.0245  0.3544  3.3693
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 17.965050   0.849663  21.144  < 2e-16 ***
## disp        -0.006622   0.004166  -1.590  0.12317
## hp          -0.022953   0.004603  -4.986 2.88e-05 ***
## wt           1.485283   0.429172   3.461  0.00175 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.062 on 28 degrees of freedom
## Multiple R-squared:  0.6808, Adjusted R-squared:  0.6466
## F-statistic: 19.91 on 3 and 28 DF,  p-value: 4.134e-07
```

Checking assumptions

Once again, before we get too eager about interpreting the model, we should check that our residuals are reasonable.

```
par(pty="s"); plot(mtc_model, which=2)
```



This Q-Q plot may lead us to question the assumption of normal errors.
That doesn't mean that we can't proceed with using our linear model, but it will mean that we should be a bit careful with how much credence we give to any quantities that depend on our normality assumption (e.g., our p-values).

Model Output: Coefficient Table

```
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 17.965050   0.849663  21.144  < 2e-16 ***
## disp        -0.006622   0.004166  -1.590  0.12317
## hp          -0.022953   0.004603  -4.986 2.88e-05 ***
## wt           1.485283   0.429172   3.461  0.00175 **
```

The intercept term and the coefficients for horsepower (`hp`) and weight (`wt`) are flagged as being significant. Thus, we would reject the null hypotheses $H_0 : \beta_0 = 0$, $H_0 : \beta_{\text{hp}} = 0$ and $H_0 : \beta_{\text{wt}} = 0$. On the other hand, there is insufficient evidence to reject the null $H_0 : \beta_{\text{wt}} = 0$.

In other words, it appears that horsepower and weight are associated with changes in quarter-mile time, but displacement is not.

Interpreting estimated coefficients I

In the case of simple linear regression, our interpretation of an estimated slope $\hat{\beta}_1$ was that an increase of one unit in our predictor was associated with an average increase of $\hat{\beta}_1$ in our response.

The interpretation is *almost* the same in multiple regression.

Interpreting estimated coefficients II

```
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 17.965050   0.849663  21.144  < 2e-16 ***
## disp        -0.006622   0.004166  -1.590  0.12317
## hp          -0.022953   0.004603  -4.986 2.88e-05 ***
## wt           1.485283   0.429172   3.461  0.00175 **
```

Our estimate for the `wt` coefficient is about 1.5.

Because we also have coefficients corresponding to engine size (`disp`) and horsepower (`hp`), our estimated coefficient of `wt` is the average increase in `qsec` associated with a unit increase of `wt` *while holding `hp` and `disp` fixed*.

Typically, say something like "controlling for `hp` and `disp`, a unit increase of `wt` is associated with an average increase of 1.5 in `qsec`".

Assessing model fit

How do we tell if our model fit is good? Let's consider the most obvious answer to this question.

We fit our model to the data by minimizing the sum of squares (this is the loss function for simple linear regression for illustration),

$$\ell(\beta_0, \beta_1) = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 = \sum_{i=1}^{n} \left(y_i - (\beta_0 + \beta_1 x_i)\right)^2 .$$

So why not use this value to assess model fit?

Assessing model fit with RSS I

We define the residual sum of squares (RSS; also called the sum of squared errors, SSE) to be the sum of squared residuals between our model and the true responses. That is, letting $\hat{\beta}_0$ and $\hat{\beta}_1$ be our estimates of the coefficients,

$$\text{RSS} = \text{SSE} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 = \sum_{i=1}^{n} \left(y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_i) \right)^2 .$$

How do we tell whether a particular RSS is large or small?

Well, before we even get to that, there are two problems...

Problem 1: We should standardize that sum

Otherwise, models fit with more observations will tend to have larger RSS. Further, if we have more parameters (i.e., more coefficients, i.e., more predictors) in our model, we are going to be able to reduce the RSS.

The important point for now is that instead of looking at the RSS, we adjust the RSS by dividing it by the *degrees of freedom* of our model: $n - (p + 1)$:

$$\frac{\text{RSS}}{\text{df}} = \frac{1}{n - (p + 1)} \sum_{i=1}^{n} (\hat{y}_i - y_i)^2$$

Generally speaking, our degrees of freedom are the number of data points, less the number of parameters: $n - (p + 1)$, if p is the number of predictors (and an additional 1 for the intercept term). So a model with more parameters will have a smaller denominator in that expression, and will have a larger RSS. That is, the denominator is smaller when we have more parameters available to our model.

Problem 2: Units of RSS are not interpretable

RSS is a sum of squares. So, like a variance, the units are nonsensical. So let's take the square root of our (standardized) RSS:

$$\sqrt{\frac{\text{RSS}}{\text{df}}} = \sqrt{\frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{n-(p+1)}}$$

This quantity is the *residual standard error* (RSE), and it is reported in the chunk of information at the bottom of our model summary:

```
## Residual standard error: 1.062 on 28 degrees of freedom
```

The degrees of freedom of a model (typically) will be the number of data points less the number of parameters we estimate. In this case, there are 32 data points and our model has four parameters.

```r
nrow(mtcars)
## [1] 32
length(coef(mtc_model))
## [1] 4
```

Assessign Model Fit: R^2

Recall the coefficient of determination, or R-squared:

$$R^2 = \frac{\text{TSS} - \text{RSS}}{\text{TSS}} = 1 - \frac{\text{RSS}}{\text{TSS}},$$

where $\text{TSS} = \sum_{i=1}^{n} (y_i - \bar{y})^2$ is the **total sum of squares** and $\text{RSS} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$ is the residual sum of squares.

If we think of

1. RSS as being the amount of variation in the data *not* captured by our model, and
2. TSS as being the amount of variation in the data (once we get rid of the structure explained by the "dumbest" model),

then $1 - \text{RSS} / \text{TSS}$ is the proportion of the variation that *is* explained by our model.

$R^2 = r^2$, where r is the correlation coefficient between the $\hat{y}_i$ and y_i:

When R^2 is close to 1, we can be confident that our linear model is accurately capturing the structure (i.e., variation) in the data.

Assessing fit another way: MSS I

Let's consider a different kind of sum of squares: the sum of squares between *our model* and the "dumbest" model:

$$\sum_{i=1}^{n} (\hat{y}_i - \bar{y})^2 .$$

This is the *model sum of squares* (MSS). This measures the amount of variation in the data explained by our model– it's like a variance of our model's predictions.

It turns out that

$$\text{TSS} = \text{RSS} + \text{MSS},$$

so that

$$R^2 = \frac{\text{TSS} - \text{RSS}}{\text{TSS}} = \frac{\text{MSS}}{\text{TSS}},$$

So we can interpret R^2 as measuring the proportion (between 0 and 1) of the variation in the responses (TSS) that is explained by our model.

Assessing fit another way: MSS II

If our model is a good fit to the data, the MSS should be large, while the RSS should be small.

So a sensible number to look at is the ratio of these two different sums of squared errors, each adjusted for their degrees of freedom:

$$\frac{\text{MSS}/p}{\text{RSS}/(n-p-1)}.$$

In fact, in the setting where our noise terms ϵ_i are independent normals with shared mean 0 and shared variance σ^2, this ratio follows a specific distribution: the F-distribution

If we look at the very bottom of our model summary, we'll see a mention of an F-statistic:

```
## F-statistic: 19.91 on 3 and 28 DF,  p-value: 4.134e-07
```

Assessing model fit with the F-distribution I

The F-distribution is a random variable distribution that arise when we look at a *ratio* of sums of squares, or a *ratio* of variances.

Its behavior is governed by two degree-of-freedom parameters (numerator and denominator), one describing the numerator and one describing the denominator.

In our case, we are looking at the ratio

$$\frac{\text{MSS}/p}{\text{RSS}/(n-p-1)} = \frac{\frac{1}{p}\sum_{i=1}^{n}\left(\hat{y}_i - \bar{y}\right)^2}{\frac{1}{n-p-1}\sum_{i=1}^{n}\left(y_i - \hat{y}_i\right)^2},$$

which is a ratio of "means" of squared errors (i.e., variance-like things!). We can show that this ratio is equivalent to a ratio of sample variances.

Assessing model fit with the F-distribution II

The important part is that under the null hypothesis in which all of the coefficients predictors are zero,

$$H_0 : \beta_1 = \cdots = \beta_p = 0,$$

our ratio of squared error terms will follow an F-distribution with parameters given by df1 $= p$ and df2 $= (n - p - 1)$.

Knowing this distribution, we can test the null hypothesis above, and associate a p-value with the null hypothesis that our model has *no explanatory power*.

```
## F-statistic: 19.91 on 3 and 28 DF,  p-value: 4.134e-07
```

the F-statistic associated with our residuals has a very small p-value associated to it, indicating, in essence, that our model fit is much better than would be expected by chance. We can be fairly certain that our model has captured a trend present in our data.

Handling categorical predictors I

Suppose that our long-standing client Catamaran wants to predict spending habits of its customers. For each customer in their database, they know whether or not that customer owns one or more cats, and whether or not they own one or more dogs. This indication of whether or not a customer owns cats (or dogs) is categorical, and it isn't obvious at first how to incorporate this information into a linear regression model.

The trick is to do something very simple: let's include two predictors:

$$x_{i,1} = \begin{cases} 1 & \text{if customer } i \text{ owns one or more cats} \\ 0 & \text{otherwise.} \end{cases}$$

and

$$x_{i,2} = \begin{cases} 1 & \text{if customer } i \text{ owns one or more dogs} \\ 0 & \text{otherwise.} \end{cases}$$

Handling categorical predictors II

Our model might be

$$y_i = \beta_0 + \beta_1 x_{i,1} + \beta_2 x_{i,2} + \epsilon_i.$$

Our predicted spending by a customer who owns cats but no dogs would be $\beta_0 + \beta_1$,

while our prediction for a customer who owns dogs but not cats would be $\beta_0 + \beta_2$,

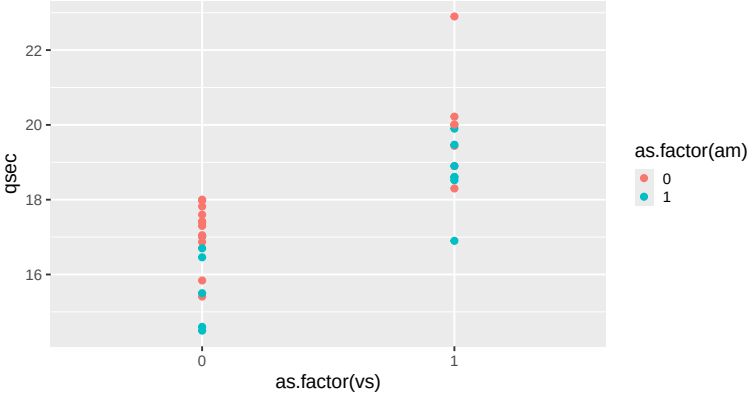
and our prediction for a customer who owns both cats and dogs would be $\beta_0 + \beta_1 + \beta_2$.

These binary variables encoding our categorical variables are often called *dummy* variables.

Our interpretation of the coefficients as "the average change in response to a unit increase" can be retained: β_1 is the (expected) change in spending associated with a "unit increase" in the "customer owns cats" variable. A unit increase in this variable (i.e., from 0 to 1) is just going from "no cats" to "owns one or more cats".

Example: `mtcars` revisited. I

Returning to the `mtcars` dataset, notice that there are a couple of binary predictors: `vs` (the engine shape, i.e., cylinder configuration; `0` for "V"-shaped, `1` for "straight") and `am` (transmission type; `0` for automatic, `1` for manual). Let's plot.



2 Categorical Predictors I

Let's see what happens when we fit a model using these two binary predictors.

```
## Call:
##  lm(formula = qsec ~ 1 + am + vs, data = mtcars)
##
##  Residuals:
##       Min        1Q    Median        3Q       Max
##  -1.77902 -0.37789  0.01687  0.65157  2.91187
##
##  Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
##  (Intercept)  17.1303     0.2754  62.195  < 2e-16 ***
##  am           -1.3091     0.3791  -3.453  0.00172 **
##  vs            2.8579     0.3753   7.614 2.15e-08 ***
##  ---
##  Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Looks like both engine shape and transmission type are useful predictors!

2 Categorical Predictors II

What if we have a category with more than two values (formally 'levels'), like demographic information? For example, state, county, race, marriage status, etc.

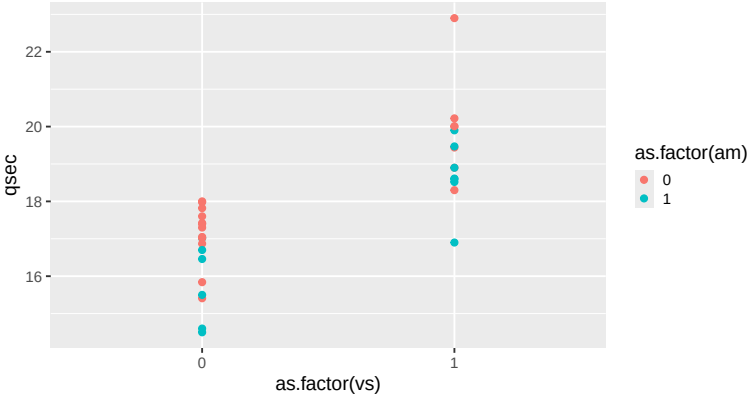
Bad Way: Assign numerical values to each level; WI=1, MI=2, IL=3, etc.

Better Way: Create one dummy variable for every level

Best Way: Choose one level / value as the "base level", and then create a dummy variable for every other level.

Interactions I

Let's look at that `mtcars` plot again with our two binary predictors.



Is it possible that when `vs=0`, the cars with `am=0` have better `qsec` than those with `am=1`, but that the transmission type doesn't make very much difference for `qsec` among the cars with `vs=1` (i.e., straight cylinder configuration).

Interactions II

This is an example of an *interaction*: the effect of one predictor seems to depend on another predictor. It's a non-linear relationship between the response and the predictors.

Certain medications will have much stronger or much different effects if they are taken in combination with other medications. For example, people taking Warfarin, a blood thinner invented here at UW-Madison, need to be very careful when taking NSAIDs such as ibuprofen, because the two drugs interact to cause gastrointestinal bleeding. Either of these two medications in isolation does not drastically increase this risk, but they *interact* to yield a much higher risk.

Interactions III

How might we include this interaction in our model? Well, the natural way is to create another predictor, given by the *product* of these two predictors:

$$y_i = \beta_0 + \beta_1 x_{i,1} + \beta_2 x_{i,2} + \beta_{1,2} x_{i,1} x_{i,2}.$$

We can include interactions in our linear models in R by writing the interaction into our model formula. We just write a colon (`:`) between the two predictors whose interaction we want to include:

```
mtc_interact <- lm( qsec ~ 1 + vs + am + vs:am, data=mtcars )
```

```
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
##  (Intercept) 17.14250    0.30497  56.210  < 2e-16 ***
##  vs           2.82464    0.50245   5.622 5.09e-06 ***
##  am          -1.34583    0.52823  -2.548   0.0166 *
##  vs:am        0.07869    0.77325   0.102   0.9197
```

Looks like in this case, what looked like an interaction to my eye turned out not to be.

Example: Toppings I

Here an interaction *is* present. This example is due to Jim Frost.

Let's consider a survey in which we ask people about how much they enjoy different foods and different condiments, and we want to build a model to predict how much people will enjoy their meal given a certain food and a certain condiment. That is, our model looks like

$$\text{satisfaction} = \beta_0 + \beta_1 * \text{food} + \beta_2 * \text{condiment}.$$

For simplicity, let's assume that our predictors `food` and `condiment` each only take two values:

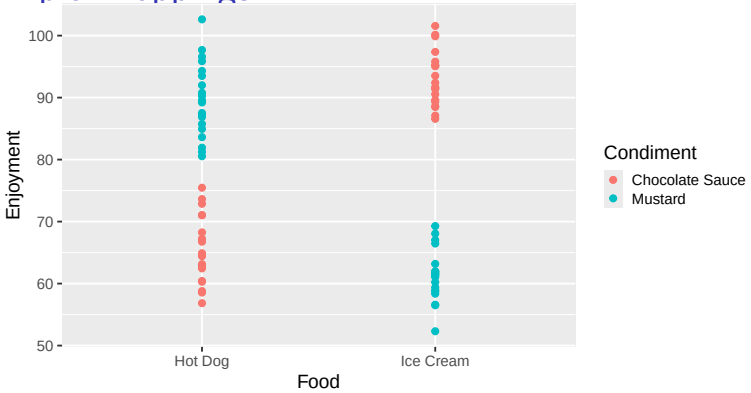
$$\text{food} \in \{\text{hot dog}, \text{ice cream}\} \quad \text{and} \quad \text{condiment} \in \{\text{mustard}, \text{chocolate}\}.$$

Example: Toppings II

Now, someone who gives a high rating for the combination of hot dogs and mustard, and a high rating for ice cream and chocolate *may not* be so enthusiastic about ice cream with mustard or a hot dog with chocolate. In other words, whether the addition of a condiment increases or decreases a person's enjoyment of a food *depends* on the value of the food predictor.

This "it depends" property is almost the definition of an interaction effect.

Example: Toppings III



It is pretty clear than when `Food=Hot Dog`, changing the `Condiment` predictor from `Chocolate` to `Mustard` increases the `Enjoyment` response, while the opposite pattern holds when `Food=Ice Cream`.

Example: Toppings IV

Let's wee what happens if we ignore this interaction:

```
## Call:
## lm(formula = Enjoyment ~ 1 + Food + Condiment, data = frost)
##
## Coefficients:
##                   Estimate Std. Error t value Pr(>|t|)
## (Intercept)        79.3237     2.9278  27.093   <2e-16 ***
## FoodIce Cream      -0.2826     3.3807  -0.084    0.934
## CondimentMustard   -3.7251     3.3807  -1.102    0.274
```

Neither the `Food` nor the `Condiment` predictor has a significant effect!

Aside: note that R has changed the names of our predictors a bit! `lm()` noticed that we had passed categorical data (e.g., `Hot Dog`, `Ice Cream`) into the linear regression, and it automatically created a dummy encoding for it. The name `FoodIce Cream` indicates that R created a dummy variable that is 1 when `Food=Ice Cream` and 0 when `Food=Hot Dog`.

Example: Toppings V

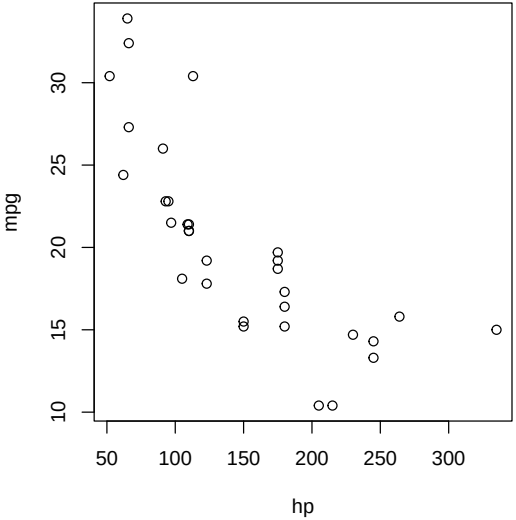
Now, let's include the interaction effect:

```
## Call:
## lm(formula = Enjoyment ~ 1 + Food + Condiment + Food:Condiment,
##     data = frost)
##
## Coefficients:
##                                Estimate Std. Error t value Pr(>|t|)
## (Intercept)                      65.317      1.120   58.34   <2e-16 ***
## FoodIce Cream                    27.731      1.583   17.52   <2e-16 ***
## CondimentMustard                 24.289      1.583   15.34   <2e-16 ***
## FoodIce Cream:CondimentMustard  -56.028      2.239  -25.02   <2e-16 ***
```

Suddenly all our predictors are significant!

Nonlinear Relationships I

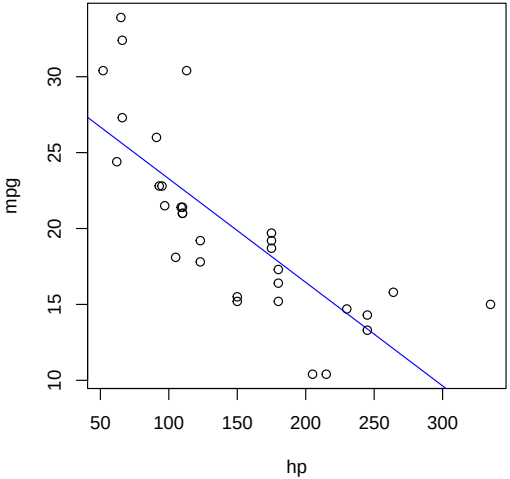
Let's come back once more to the `mtcars` data set, and let's look in particular at `mpg` (miles per gallon) and `hp` (horsepower).

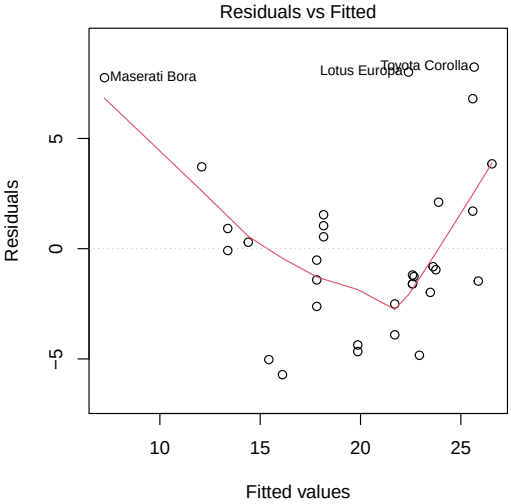


Let's fit a linear model to this data and see how things look.

```
mtc_lm <- lm( mpg ~ 1 + hp, data=mtcars );
```

Nonlinear Relationships II





Nonlinear Relationships III

Visually, it looks as though our residuals tend to be positive for small and larger values of our predictor, but negative for mid-range values.

It looks a bit like there is a non-linear trend in our data– as horsepower increases, `mpg` decreases very quickly at first, and then levels off. That isn't what we would see if the relationship were simply linear– it's more in keeping with a nonlinear trend.

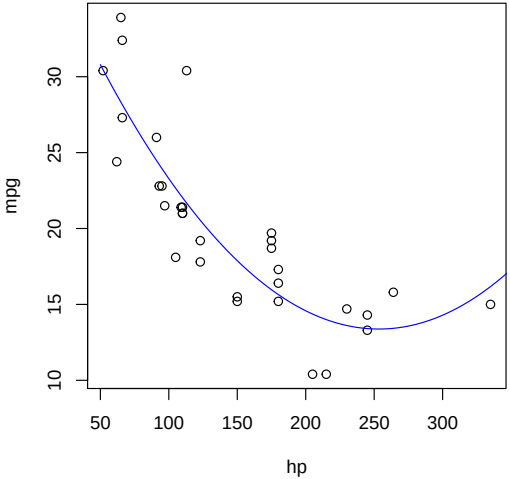
The curve looks a bit like a quadratic function. So what if we add a squared term to our predictors?

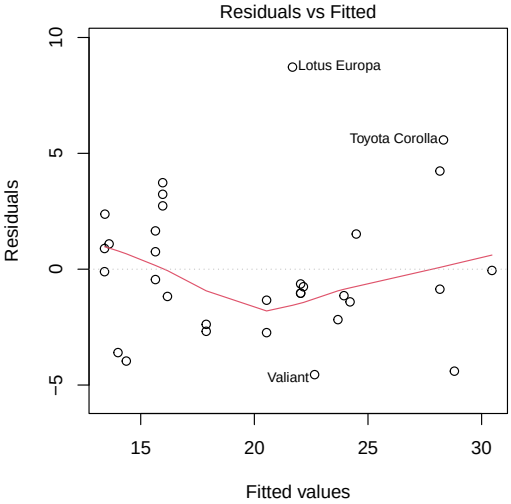
To get our non-linear term `hp^2` into the model, we have to write our formula as `mpg ~ 1 + hp + I(hp^2)`.

```
mtc_lm2 <- lm( mpg ~ 1 + hp + I(hp^2), data=mtcars );
```

If we just wrote `mpg ~ 1 + hp + hp^2`, R would parse `hp^2` as just `hp`. `I(...)` prevents this.

Nonlinear Relationships IV





Definitely a better fit!

Nonlinear Relationships V

Generally speaking, if you see a non-linear trend in the data, a nonlinear transformation is usually the easiest solution.

Often you would want to add power transformations *in addition to* the original `x` predictor, but a log or square root transformation would *replace* `x`. However, you may need to be creative.

Review

In these notes we covered

- The general multiple regression model
- Interpretation of estimated coefficients
- Residual Standard Error
- Assessing Model fit with R^2
- MSS and the F Statistic for overall fit
- Categorical predictors
- Interaction Terms
- Nonlinear Transformations